



# დავადება 4

პოლიმორფიზმი,  
მემკვიდრეობითობა,  
პრიორიტეტული რიგი

## დავალებების აღწერა

|                             |    |
|-----------------------------|----|
| §1 — Employee.h.....        | 2  |
| §2 — Employee.cpp.....      | 5  |
| §3 — WorkerClasses.h.....   | 6  |
| §4 — WorkerClasses.cpp..... | 9  |
| §5 — PriorityQueue.h.....   | 14 |
| §6 — PriorityQueue.cpp..... | 16 |
| §7 — Driver.cpp.....        | 19 |
| ტესტირების შედეგები.....    | 22 |

## თავი 1 — Employee.h

```
1  #pragma once
2  #include <iostream>
3  #include <string>
4
5  // Qualification level used by Developer / Engineer / Tester roles
6  enum class QualificationLevel { Junior, Middle, Senior };
7  // Technologies used only by Developers (Backend/Frontend)
8  enum class DevTech { DotNet, Spring, Django, Angular, React, Vue };
9
10 // Converts enum values to readable strings for printing
11 std::string QualificationToString(QualificationLevel level);
12 std::string DevTechToString(DevTech tech);
13
14 // Abstract base class that represents any employee in the company.
15 class Employee
16 {
17 protected:
18     std::string name; // Employee full name
19     int empID; // Unique employee identifier
20     std::string empClass; // Class code (CIO, PM, BD, FD, DB, DE, TST)
21     int tenureMonths; // Tenure in the company (in months)
22     double BaseSalary; // Base salary before bonuses (varies by role/level/tech)
23
24 public:
25     // Constructs an employee with all common fields.
26     Employee(const std::string& n, int id, const std::string& cls, int tenure, double Base);
27     // Virtual destructor is required because we delete through Employee* pointers.
28     virtual ~Employee();
29     // Calculates total salary for the employee (base + bonuses).
30     virtual double calcSalary() const = 0; // Pure virtual => Employee is abstract; each derived role must implement it.
31     // Prints common employee info (derived classes can extend it).
32     virtual void print(std::ostream& os) const;
33 };
34 // overloaded << operator
35 std::ostream& operator<<(std::ostream& os, const Employee& E);
```

### ზოგადი დანიშნულება

Employee.h შეიცავს საბაზო (abstract) კლასს Employee, ასევე enum-ებს QualificationLevel და DevTech და დამხმარე ფუნქციებს, რომლებიც enum მნიშვნელობებს სტრინგად გარდაქმნიან. ეს ფაილი ქმნის საერთო ინტერფეისს ყველა თანამშრომლისთვის (პოლიმორფიზმი: calcSalary() და print()).

### Enum-ები

QualificationLevel აღწერს კვალიფიკაციის დონეს იმ პოზიციებისთვის, სადაც ეს მნიშვნელოვანია (Developer/Engineer/Tester): Junior, Middle, Senior.

DevTech აღწერს ტექნოლოგიას, რომელსაც იყენებს დეველოპერი. რეალურად კონკრეტულ ქვეკლასზეა შეზღუდვა: BackendDeveloper იღებს მხოლოდ DotNet/Spring/Django, ხოლო FrontendDeveloper — Angular/React/Vue.

### std::string QualificationToString(QualificationLevel level)

დანიშნულება: QualificationLevel მნიშვნელობის ტექსტურ ფორმად გადაყვანა, რათა ბეჭდვისას მივიღოთ „Junior/Middle/Senior“.

რას აკეთებს:

- switch/case-ით აბრუნებს შესაბამის სტრინგს.

- თუ მოხვდა არასწორი მნიშვნელობა (პრაქტიკაში არ უნდა მოხდეს), აბრუნებს ცარიელ სტრინგს.

### **std::string DevTechToString(DevTech tech)**

დანიშნულება: DevTech მნიშვნელობის ტექსტურ ფორმად გადაყვანა, რათა ბეჭდვისას მივიღოთ „.Net/Spring/Django/...“.

რას აკეთებს:

- switch/case-ით აბრუნებს შესაბამის სტრინგს.
- უცნობ შემთხვევაში აბრუნებს ცარიელ სტრინგს.

### **კლასი Employee (abstract base class)**

Employee არის აბსტრაქტული კლასი, რომელიც აერთიანებს ყველა თანამშრომელს ერთ საერთო ტიპში. ამით შესაძლებელი ხდება Employee\* ტიპით სხვადასხვა ობიექტის ერთად შენახვა და პოლიმორფულად გამოძახება.

- საერთო ველები: name, empID, empClass, tenureMonths, BaseSalary.
- საერთო ქცევა: print() — ბეჭდავს საერთო ველებს.
- სავალდებულო ქცევა ქვეკლასებისთვის: calcSalary() — თითოეულ პოზიციას თავისი წესით აქვს ხელფასი.

### **Employee-ის ველები (protected)**

protected წვდომა ნიშნავს: ველებს პირდაპირ იყენებენ შთამომავალი კლასები, მაგრამ გარე კოდიდან არ ჩანს.

- name — თანამშრომლის სახელი.
- empID — უნიკალური ID.
- empClass — პოზიციის კოდი.
- tenureMonths — სტაჟი თვეებში; გამოიყენება ბონუსის გამოსათვლელად.
- BaseSalary — საბაზო ხელფასი, რომელიც შემდეგ ბონუსით იზრდება.

### **Employee::Employee(const std::string& n, int id, const std::string& cls, int tenure, double Base)**

დანიშნულება: Employee-ის საერთო კონსტრუქტორი — ავსებს ყველა საერთო ველს.

რას აკეთებს:

- ინიციალიზაციის სიაში ინახავს name/id/class/tenure/BaseSalary.
- შთამომავალი კლასები მას იძახებენ თავიანთ კონსტრუქტორებში.

### **virtual Employee::~~Employee()**

დანიშნულება: ვირტუალური დესტრუქტორი — აუცილებელია, რადგან PriorityQueue-ში ობიექტები ინახება Employee\* pointer-ებით და იშლება პოლიმორფულად.

შენიშვნები:

- თუ დესტრუქტორი ვირტუალური არ იქნებოდა, `delete Employee*` მხოლოდ `base` ნაწილის დესტრუქტორს გამოიძახებდა.

**`virtual double Employee::calcSalary() const = 0`**

დანიშნულება: სრული ხელფასის გამოთვლა.

**`virtual void Employee::print(std::ostream& os) const`**

დანიშნულება: სტანდარტული ბეჭდვა: აჩვენებს საერთო ველებს და ასევე `calcSalary()`-ის შედეგს.

**`std::ostream& operator<<(std::ostream& os, const Employee& E)`**

დანიშნულება: მოსახერხებელი ბეჭდვა „<<“-ით.

რას აკეთებს:

- იძახებს `E.print(os)`-ს, ამიტომ მუშაობს პოლიმორფულად (თუ ობიექტი რეალურად შთამომავალია).

## თავი 2 — Employee.cpp

```
1  #include "Employee.h"
2
3  // Converts QualificationLevel enum to a human-readable string
4  std::string QualificationToString(QualificationLevel level)
5  {
6      switch (level)
7      {
8          case QualificationLevel::Junior: return "Junior";
9          case QualificationLevel::Middle: return "Middle";
10         case QualificationLevel::Senior: return "Senior";
11     }
12     return ""; // Fallback (should never happen)
13 }
14 // Converts DevTech enum to a human-readable string
15 std::string DevTechToString(DevTech tech)
16 {
17     switch (tech)
18     {
19         case DevTech::DotNet: return ".Net";
20         case DevTech::Spring: return "Spring";
21         case DevTech::Django: return "Django";
22         case DevTech::Angular: return "Angular";
23         case DevTech::React: return "React";
24         case DevTech::Vue: return "Vue";
25     }
26     return ""; // Fallback (should never happen)
27 }
28 // Base constructor initializes common employee fields
29 Employee::Employee(const std::string& n, int id, const std::string& cls, int tenure, double Base)
30     : name(n), empID(id), empClass(cls), tenureMonths(tenure), BaseSalary(Base) {}
31 // Virtual destructor (needed for polymorphic deletion)
32 Employee::~Employee() {}
33 // Prints common employee information.
34 void Employee::print(std::ostream& os) const {
35     os << "Name: " << name
36         << " | ID: " << empID
37         << " | Class: " << empClass
38         << " | Tenure(months): " << tenureMonths
39         << " | Base salary: " << BaseSalary
40         << " | Total salary: " << calcSalary();
41 }
42 // overloaded << operator
43 std::ostream& operator<<(std::ostream& os, const Employee& E) {
44     E.print(os);
45     return os;
46 }
```

### ზოგადი დანიშნულება

Employee.cpp შეიცავს Employee.h-ში დეკლარირებული ფუნქციების რეალიზაციებს: QualificationToString, DevTechToString, Employee-ის კონსტრუქტორი/დესტრუქტორი, print() და operator<<.

### QualificationToString / DevTechToString

ეს ორი ფუნქცია გამოიყენება მხოლოდ გამოსატანად.

სტრინგებად გარდაქმნა მნიშვნელოვანია, რადგან enum მნიშვნელობები პირდაპირ მომხმარებლისთვის წაკითხვადი არ არის.

### Employee::print()

print() არის ერთიანი ფორმატი ყველა თანამშრომლისთვის.

## თავი 3 — WorkerClasses.h

```
1  #pragma once
2  #include "Employee.h"
3
4  // CIO role
5  class CIO : public Employee
6  {
7  public:
8      CIO(const std::string& n, int id, int tenure);
9      double calcSalary() const override; // Salary calculation for CIO
10     void print(std::ostream& os) const override; // Prints role name + base fields
11 };
12
13 // Project Manager role
14 class ProjectManager : public Employee
15 {
16 public:
17     ProjectManager(const std::string& n, int id, int tenure);
18     double calcSalary() const override;
19     void print(std::ostream& os) const override;
20 };
21
22 // Abstract idea of "Developer" in hierarchy.
23 class Developer : public Employee
24 {
25 protected:
26     QualificationLevel Level; // Junior / Middle / Senior
27     DevTech Technology; // Allowed tech depends on Backend / Frontend
28
29 public:
30     // Base developer constructor used by BackendDeveloper and FrontendDeveloper.
31     Developer(const std::string& n, int id, const std::string& cls, int tenure, QualificationLevel lvl, D
32 };
33
34 // Backend Developer: uses only (.Net, Spring, Django)
35 class BackendDeveloper : public Developer
36 {
37 public:
38     BackendDeveloper(const std::string& n, int id, int tenure, QualificationLevel lvl, DevTech tech);
39     double calcSalary() const override;
40     void print(std::ostream& os) const override;
41 };
42
43 // Frontend Developer: uses only (Angular, React, Vue)
44 class FrontendDeveloper : public Developer
45 {
46 public:
```

### ზოგადი დანიშნულება

WorkerClasses.h აღწერს Employee-ის შთამომავალ კლასებს და ორ საბაზო კლასს Developer და Engineer.

### CIO და ProjectManager

ორივე კლასი პირდაპირ ამემკვიდრებს Employee-ს.

ამ კლასებში BaseSalary ფიქსირებული და tenureMonths-ზე დამოკიდებული ბონუსი ემატება.

### Developer

Developer აკავშირებს დეველოპერის საერთო ატრიბუტებს: QualificationLevel და DevTech. ჰყავს ორი მემკვიდრე: BackendDeveloper და FrontendDeveloper



## BackendDeveloper და FrontendDeveloper

ორივე ამემკვიდრებს Developer-ს და აიმპლემენტებს calcSalary() და print()-ს.

კონსტრუქტორში ხდება ტექნოლოგიის ვალიდაცია და BaseSalary-ის დათვლა level+tech-ის მიხედვით.

- BackendDeveloper: დასაშვებია მხოლოდ DotNet/Spring/Django.
- FrontendDeveloper: დასაშვებია მხოლოდ Angular/React/Vue.

## Engineer

Engineer ინახავს მხოლოდ QualificationLevel-ს, რადგან ტექნოლოგია ამ ტიპებში არ გაქვს მითითებული დავალების მიხედვით. ჰყავს ორი შვილი: DataBase Engineer და DevOps Engineer.

## DataBaseEngineer და DevOpsEngineer

ორივე ამემკვიდრებს Engineer-ს.

კონსტრუქტორში base salary დონის მიხედვით იანგარიშება, ხოლო calcSalary-ში tenure-ის ბონუსი ემატება.

## Tester

Tester პირდაპირ Employee-დან მემკვიდრეობს, მაგრამ აქვს საკუთარი QualificationLevel ველი.



## တစ်ခိုဒ် 4 — WorkerClasses.cpp

```
1  #include <stdexcept>
2  #include "WorkerClasses.h"
3
4  // ----- CIO -----
5
6  CIO::CIO(const std::string& n, int id, int tenure)
7  : Employee(n, id, "CIO", tenure, 10000.0) {
8
9  // CIO salary: base + prorated yearly bonus (depends on tenure)
10
11 double CIO::calcSalary() const {
12     double YearlyBonus = 1200.0 * (tenureMonths / 12.0);
13     return BaseSalary + YearlyBonus;
14 }
15
16 void CIO::print(std::ostream& os) const {
17     os << "[CIO]\n";
18     Employee::print(os);
19 }
20
21 // ----- Project Manager -----
22
23 ProjectManager::ProjectManager(const std::string& n, int id, int tenure)
24 : Employee(n, id, "PM", tenure, 7000.0) {
25
26 // PM salary: base + prorated yearly bonus
27
28 double ProjectManager::calcSalary() const {
29     double YearlyBonus = 1000.0 * (tenureMonths / 12.0);
30     return BaseSalary + YearlyBonus;
31 }
32
33 void ProjectManager::print(std::ostream& os) const {
34     os << "[Project Manager]\n";
35     Employee::print(os);
36 }
37
38 // ----- Developer base class -----
39
40 Developer::Developer(const std::string& n, int id, const std::string& cls, int tenure, QualificationLevel lvl, DevTech tech, double Base)
41 : Employee(n, id, cls, tenure, Base), Level(lvl), Technology(tech) {}
42
43 // ----- Backend Developer -----
44
45 BackendDeveloper::BackendDeveloper(const std::string& n, int id, int tenure, QualificationLevel lvl, DevTech tech)
46 : Developer(n, id, "BD", tenure, lvl, tech, 0.0)
47 {
48     // Validate: backend developer is allowed only these technologies
49     if (!(tech == DevTech::DotNet || tech == DevTech::Spring || tech == DevTech::Django)) {
50         throw std::invalid_argument(
51             "ERROR: BackendDeveloper can use only (.Net, Spring, Django). Given: " + DevTechToString(tech));
52     }
53
54 // Base salary depends on qualification level
55 double Base = (lvl == QualificationLevel::Junior) ? 4000.0 :
56               (lvl == QualificationLevel::Middle) ? 5500.0 : 7000.0;
57
58 // Extra salary depends on specific backend technology
59 if (tech == DevTech::DotNet) Base += 150.0;
60 else if (tech == DevTech::Spring) Base += 300.0;
61 else if (tech == DevTech::Django) Base += 200.0;
62
63 // Store computed base salary into Employee field
64 BaseSalary = Base;
65
66 // Backend developer salary: base + prorated yearly bonus
67
68 double BackendDeveloper::calcSalary() const {
69     double YearlyBonus = 840.0 * (tenureMonths / 12.0);
70     return BaseSalary + YearlyBonus;
71 }
72
73 void BackendDeveloper::print(std::ostream& os) const {
74     os << "[Backend Dev]\n";
75     Employee::print(os);
76     os << " | Level: " << QualificationToString(Level)
77       << " | Tech: " << DevTechToString(Technology);
78 }
79
80 // ----- Frontend Developer -----
81
82 FrontendDeveloper::FrontendDeveloper(const std::string& n, int id, int tenure, QualificationLevel lvl, DevTech tech)
83 : Developer(n, id, "FD", tenure, lvl, tech, 0.0)
84 {
85     // Validate: frontend developer is allowed only these technologies
86     if (!(tech == DevTech::Angular || tech == DevTech::React || tech == DevTech::Vue)) {
87         throw std::invalid_argument(
88             "ERROR: FrontendDeveloper can use only (Angular, React, Vue). Given: " + DevTechToString(tech));
89     }
90
91 // Base salary depends on qualification level
92 double Base = (lvl == QualificationLevel::Junior) ? 3800.0 :
93               (lvl == QualificationLevel::Middle) ? 5200.0 : 6600.0;
94
95 // Extra salary depends on specific frontend technology
96 if (tech == DevTech::Angular) Base += 150.0;
97 else if (tech == DevTech::React) Base += 250.0;
98 else if (tech == DevTech::Vue) Base += 180.0;
```

```

128 // ----- DevOps Engineer -----
129
130 DevOpsEngineer::DevOpsEngineer(const std::string& n, int id, int tenure, QualificationLevel lvl)
131     : Engineer(n, id, "DE", tenure, lvl, 0.0)
132 {
133     // Base salary depends on qualification level
134     double Base = (lvl == QualificationLevel::Junior) ? 4600.0 :
135         (lvl == QualificationLevel::Middle) ? 6200.0 : 7800.0;
136
137     BaseSalary = Base;
138 }
139
140 double DevOpsEngineer::calcSalary() const {
141     double YearlyBonus = 1020.0 * (tenureMonths / 12.0);
142     return BaseSalary + YearlyBonus;
143 }
144
145 void DevOpsEngineer::print(std::ostream& os) const {
146     os << "[DevOps Engineer]\n";
147     Employee::print(os);
148     os << " | Level: " << QualificationToString(Level);
149 }
150
151 // ----- Tester -----
152
153 Tester::Tester(const std::string& n, int id, int tenure, QualificationLevel lvl)
154     : Employee(n, id, "TST", tenure, 0.0), Level(lvl)
155 {
156     // Base salary depends on qualification level
157     double Base = (lvl == QualificationLevel::Junior) ? 3000.0 :
158         (lvl == QualificationLevel::Middle) ? 4000.0 : 5000.0;
159
160     BaseSalary = Base;
161 }
162
163 double Tester::calcSalary() const {
164     double YearlyBonus = tenureMonths * 40.0;
165     return BaseSalary + YearlyBonus;
166 }
167
168 void Tester::print(std::ostream& os) const {
169     os << "[Tester]\n";
170     Employee::print(os);
171     os << " | Level: " << QualificationToString(Level);
172 }

```

```

88     BaseSalary = Base;
89 }
90
91 double FrontendDeveloper::calcSalary() const {
92     double YearlyBonus = 780.0 * (tenureMonths / 12.0);
93     return BaseSalary + YearlyBonus;
94 }
95
96 void FrontendDeveloper::print(std::ostream& os) const {
97     os << "[Frontend Dev]\n";
98     Employee::print(os);
99     os << " | Level: " << QualificationToString(Level)
100     << " | Tech: " << DevTechToString(Technology);
101 }
102
103 // ----- Engineer base class -----
104
105 Engineer::Engineer(const std::string& n, int id, const std::string& cls, int tenure, QualificationLevel lvl, double Base)
106     : Employee(n, id, cls, tenure, Base), Level(lvl) {}
107
108 // ----- Database Engineer -----
109
110 DataBaseEngineer::DataBaseEngineer(const std::string& n, int id, int tenure, QualificationLevel lvl)
111     : Engineer(n, id, "DB", tenure, lvl, 0.0)
112 {
113     // Base salary depends on qualification level
114     double Base = (lvl == QualificationLevel::Junior) ? 4500.0 :
115         (lvl == QualificationLevel::Middle) ? 6000.0 : 7500.0;
116
117     BaseSalary = Base;
118 }
119
120 double DataBaseEngineer::calcSalary() const {
121     double YearlyBonus = 960.0 * (tenureMonths / 12.0);
122     return BaseSalary + YearlyBonus;
123 }
124
125 void DataBaseEngineer::print(std::ostream& os) const {
126     os << "[DB Engineer]\n";
127     Employee::print(os);
128     os << " | Level: " << QualificationToString(Level);
129 }
130
131 // ----- DevOps Engineer -----
132
133 DevOpsEngineer::DevOpsEngineer(const std::string& n, int id, int tenure, QualificationLevel lvl)
134     : Engineer(n, id, "DE", tenure, lvl, 0.0)
135 {
136     // Base salary depends on qualification level

```

## ზოგადი დანიშნულება

WorkerClasses.cpp შეიცავს კონკრეტული პოზიციების რეალიზაციებს: კონსტრუქტორებს, calcSalary()-ებს და print()-ებს. ამ ფაილში ჩანს ხელფასის დათვლის წესები: პოზიცია + (Level/Tech თუ საჭიროა) + სტაჟი (tenureMonths).

## საერთო პრინციპი ხელფასის გამოთვლაში

ყველა პოზიცია იყენებს BaseSalary-ს როგორც საფუძველს.

შემდეგ ემატება ბონუსი, რომელიც დამოკიდებულია tenureMonths-ზე.

Developer/Engineer/Testers-ში BaseSalary ასევე დამოკიდებულია qualification level-ზე (და developer-ში tech-ზე).

## CIO::calcSalary()

დანიშნულება: CIO-ის სრული ხელფასი.

რას აკეთებს:

- $\text{YearlyBonus} = 1200.0 * (\text{tenureMonths} / 12.0)$  — ბონუსი პროპორციულია კომპანიაში გატარებულ წელთან.
- აბრუნებს  $\text{BaseSalary} + \text{YearlyBonus}$ .

## ProjectManager::calcSalary()

დანიშნულება: Project Manager-ის სრული ხელფასი.

რას აკეთებს:

- $\text{YearlyBonus} = 1080.0 * (\text{tenureMonths} / 12.0)$ .
- აბრუნებს  $\text{BaseSalary} + \text{YearlyBonus}$ .

## BackendDeveloper constructor tech validation

დანიშნულება: BackendDeveloper-ის კონსტრუქტორში ტექნოლოგიის შემოწმების შემოწმება.

რას აკეთებს:

- თუ tech არაა DotNet/Spring/Django -> throw std::invalid\_argument.
- ეს იცავს ობიექტის ლოგიკურ სისწორეს (backend დეველოპერს frontend tech არ უნდა ჰქონდეს).

## BackendDeveloper BaseSalary calculation

დანიშნულება: BackendDeveloper-ის BaseSalary-ის გამოთვლა level+tech-ის მიხედვით.

რას აკეთებს:

- Level-ზე დამოკიდებული ძირითადი ზღვარი: Junior=4000, Middle=5500, Senior=7000.
- Tech-ზე დამოკიდებული დანამატი: DotNet +150, Spring +300, Django +200.

- BaseSalary ინახება Employee-ის ველში.

### **BackendDeveloper::calcSalary()**

დანიშნულება: BackendDeveloper-ის სრული ხელფასი.

რას აკეთებს:

- $\text{YearlyBonus} = 840.0 * (\text{tenureMonths} / 12.0)$ .
- აბრუნებს BaseSalary + YearlyBonus.

### **FrontendDeveloper constructor tech validation**

დანიშნულება: FrontendDeveloper-ის კონსტრუქტორში ტექნოლოგიის შეზღუდვის შემოწმება.

რას აკეთებს:

- თუ tech არაა Angular/React/Vue -> throw std::invalid\_argument.

### **FrontendDeveloper BaseSalary calculation**

დანიშნულება: FrontendDeveloper-ის BaseSalary-ის გამოთვლა level+tech-ის მიხედვით.

რას აკეთებს:

- Level-ის საფუძვლები: Junior=3800, Middle=5200, Senior=6600.
- Tech დანამატი: Angular +150, React +250, Vue +180.

### **Engineer base class**

დანიშნულება: Engineer-ის კონსტრუქტორი ინახავს Level-ს და ავსებს Employee-ის საერთო ველებს.

შენიშვნები:

- Engineer-ში არ ხდება BaseSalary-ის დათვლა — ამას აკეთებენ DataBaseEngineer/DevOpsEngineer თავიანთ კონსტრუქტორებში.

### **DataBaseEngineer::calcSalary()**

დანიშნულება: DataBase Engineer-ის სრული ხელფასი.

რას აკეთებს:

- BaseSalary დონის მიხედვით დათვლილია კონსტრუქტორში.
- $\text{YearlyBonus} = 960.0 * (\text{tenureMonths} / 12.0)$ .

### **DevOpsEngineer::calcSalary()**

დანიშნულება: DevOps Engineer-ის სრული ხელფასი.

რას აკეთებს:

- BaseSalary დონის მიხედვით დათვლილია კონსტრუქტორში.
- $\text{YearlyBonus} = 1020.0 * (\text{tenureMonths} / 12.0)$ .

## **Tester::calcSalary()**

დანიშნულება: Tester-ის სრული ხელფასი.

რას აკეთებს:

- BaseSalary დონის მიხედვით დათვლილია კონსტრუქტორში.
- ბონუსი: tenureMonths \* 40.0.
- აბრუნებს BaseSalary + ბონუსს.



## თავი 5 — PriorityQueue.h

```
1  #pragma once
2  #include <iostream>
3  #include "Employee.h"
4
5  // Max-heap Priority Queue where priority is Employee salary.
6  class PriorityQueue
7  {
8  private:
9      Employee** Data; // Dynamic array of pointers to Employee objects (heap storage)
10     int Capacity; // Maximum number of employees the queue can store
11     int size; // Current number of employees in the heap
12
13     // Restores heap property after insertion
14     void siftUp(int index);
15     // Restores heap property after deletion
16     void siftDown(int index);
17     // Index helpers for binary heap
18     int Parent(int index) const;
19     int LeftChild(int index) const;
20     int RightChild(int index) const;
21
22 public:
23
24     PriorityQueue(int cap = 100); // Creates a priority queue with given capacity (default 100)
25     ~PriorityQueue(); // Deletes remaining employees still stored in the queue and frees array memory
26     bool isEmpty() const; // Returns true if there are no employees in the queue
27     bool isFull() const; // Returns true if the queue has reached its capacity
28     int getSize() const; // Returns current number of employees
29     void insert(Employee* E); // Inserts an employee pointer into the heap
30     Employee* peekMax() const; // Returns the highest paid employee
31     Employee* extractMax(); // Removes and returns the highest paid employee
32     void print(std::ostream& os) const; // Prints all employees currently stored in heap order
33 };
```

### ზოგადი დანიშნულება

PriorityQueue.h აღწერს მასივით იმპლემენტირებულ პრიორიტეტულ რიგს (Max-Heap). პრიორიტეტი არის თანამშრომლის სრული ხელფასი (calcSalary()).

ყველაზე მაღალი ხელფასის მქონე თანამშრომელი ყოველთვის არის heap-ის root-ზე (index 0).

### მონაცემები

Queue ინახავს Employee\* pointer-ებს, რადგან რეალურად ობიექტები პოლიმორფულია (სხვადასხვა ქვეკლასი).

- Data — დინამიკური მასივი Employee\* ტიპის ელემენტებით (heap storage).
- Capacity — მაქსიმალური რაოდენობა.
- size — ელემენტების მიმდინარე რაოდენობა.

### დამხმარე ფუნქციები heap-ისთვის

Parent/LeftChild/RightChild აბრუნებენ ინდექსებს 0-based heap ფორმულის მიხედვით.

siftUp და siftDown ასწორებენ heap property-ს, რათა insert/extractMax მუშაობდეს სწორად.

### **insert(Employee\* E)**

დანიშნულება: ამატებს ახალ თანამშრომელს heap-ში (პრიორიტეტი: calcSalary()).

რას აკეთებს:

- თუ რიგი სავსეა -> throw std::runtime\_error.
- ელემენტი დგება მასივის ბოლოში (index = size).
- შემდეგ siftUp(size) ასწორებს heap-ს.
- ბოლოს size იზრდება.

### **peekMax()**

დანიშნულება: აბრუნებს (არ შლის) ყველაზე მაღალი ხელფასის მქონე თანამშრომელს.

რას აკეთებს:

- თუ რიგი ცარიელია -> nullptr.
- სხვა შემთხვევაში Data[0] (root).

### **extractMax()**

დანიშნულება: შლის და აბრუნებს ყველაზე მაღალი ხელფასის მქონე თანამშრომელს.

რას აკეთებს:

- თუ ცარიელია -> nullptr.
- ინახავს root pointer-ს (maxEmp).
- root-ზე გადმოაქვს ბოლო ელემენტი.
- size მცირდება.
- siftDown(0) აღადგენს heap property-ს.
- აბრუნებს maxEmp-ს.

### **print(std::ostream& os)**

დანიშნულება: ბეჭდავს heap-ში არსებული ელემენტებს მასივის მიმდინარე რიგითობით.

შენიშვნები:

- ეს არ არის „სორტირებული“ ბეჭდვა; ეს არის heap array order.
- თითოეული Employee\* თავის print()-ს იყენებს პოლიმორფულად.



## တၢ်ဒၣ် 6 — PriorityQueue.cpp

```
1  #include <utility>
2  #include <stdexcept>
3  #include "PriorityQueue.h"
4
5  // Allocates dynamic array for heap storage
6  PriorityQueue::PriorityQueue(int cap) : Data(nullptr), Capacity(cap), size(0)
7  {
8      Data = new Employee * [Capacity];
9  }
10 // Destructor: deletes all Employee objects still inside the queue, then deletes the array itself
11 PriorityQueue::~PriorityQueue() {
12     for (int i = 0; i < size; i++) {
13         delete Data[i];
14     }
15     delete[] Data;
16 }
17
18 bool PriorityQueue::isEmpty() const { return size == 0; }
19 bool PriorityQueue::isFull() const { return size == Capacity; }
20 int  PriorityQueue::getSize() const { return size; }
21
22 // Heap index calculations
23 int PriorityQueue::Parent(int index) const { return (index - 1) / 2; }
24 int PriorityQueue::LeftChild(int index) const { return 2 * index + 1; }
25 int PriorityQueue::RightChild(int index) const { return 2 * index + 2; }
26
27 // Moves element at 'index' upward until heap property is restored
28 void PriorityQueue::siftUp(int index)
29 {
30     while (index > 0) {
31         int p = Parent(index);
32         // Compare by total salary (priority)
33         if (Data[p]->calcSalary() < Data[index]->calcSalary()) {
34             std::swap(Data[p], Data[index]);
35             index = p;
36         }
37         else break;
38     }
39 }
40 // Moves element at 'index' downward until heap property is restored
41 void PriorityQueue::siftDown(int index)
42 {
43     while (true) {
44         int Left = LeftChild(index);
45         int Right = RightChild(index);
46         int Largest = index;
47     }
```

```

47
48 // Find the child with larger salary priority
49 if (Left < size && Data[Left]->calcSalary() > Data[Largest]->calcSalary())
50     Largest = Left;
51 if (Right < size && Data[Right]->calcSalary() > Data[Largest]->calcSalary())
52     Largest = Right;
53 // If a child is larger, swap and continue
54 if (Largest != index) {
55     std::swap(Data[index], Data[Largest]);
56     index = Largest;
57 }
58 else break;
59 }
60 }
61 // Inserts new element and restores heap property
62 void PriorityQueue::insert(Employee* E)
63 {
64     if (isFull())
65         throw std::runtime_error("PriorityQueue is full - cannot insert new employee");
66
67     Data[size] = E; // Place at the end
68     siftUp(size); // Restore heap property
69     size++; // Increase size
70 }
71 // Returns max element
72 Employee* PriorityQueue::peekMax() const
73 {
74     if (isEmpty()) return nullptr;
75     return Data[0];
76 }
77 // Removes max element, restores heap, returns removed pointer.
78 Employee* PriorityQueue::extractMax()
79 {
80     if (isEmpty()) return nullptr;
81
82     Employee* maxEmp = Data[0]; // Store max element
83     Data[0] = Data[size - 1]; // Move last to root
84     size--; // Shrink heap
85     if (!isEmpty()) siftDown(0); // Restore heap property
86     return maxEmp; // Return removed element (not deleted here!)
87 }
88
89 // Prints elements in heap array order
90 // Each employee prints their own role + fields polymorphically
91 void PriorityQueue::print(std::ostream& os) const
92 {
93     for (int i = 0; i < size; ++i) {
94         os << "\n[" << (i + 1) << " ] ";
95         Data[i]->print(os);
96         os << '\n';
97     }
98 }

```

## ზოგადი დანიშნულება

PriorityQueue.cpp შეიცავს heap ალგორითმების რეალიზაციას. მთავარი იდეა: Data მასივი წარმოადგენს სრულ ბინარულ ხეს, სადაც ყოველი კვანძის ხელფასი  $\geq$  მისი შვილების ხელფასს.

## კონსტრუქტორი და დესტრუქტორი

PriorityQueue(int cap) გამოყოფს Employee\* მასივისთვის მეხსიერებას.

დესტრუქტორი წმენდს დარჩენილ ელემენტებს (delete Data[i]) და შემდეგ delete[] Data.

- ეს ნიშნავს: თუ რაიმე Employee\* რიგში დარჩა პროგრამის ბოლოს, memory leak არ იქნება.
- მაგრამ extractMax()-ით ამოღებულ ელემენტს დესტრუქტორი აღარ შეეხება (რადგან size შემცირდა) — ამიტომ Driver.cpp-ში delete აუცილებელია.

### Parent/LeftChild/RightChild

დანიშნულება: Heap index helper-ები 0-based მასივისთვის.

რას აკეთებს:

- $\text{Parent}(i) = (i-1)/2$  — მშობლის ინდექსი.
- $\text{LeftChild}(i) = 2*i+1$  — მარცხენა შვილის ინდექსი.
- $\text{RightChild}(i) = 2*i+2$  — მარჯვენა შვილის ინდექსი.

### siftUp(int index)

დანიშნულება: insert-ის შემდეგ ელემენტი ადის ზემოთ სანამ heap წესს არ დააკმაყოფილებს.

რას აკეთებს:

- ადარებს Data[index] და მის Parent-ს calcSalary()-ით.
- თუ ბავშვი უფრო დიდია, ხდება swap და index ხდება parent.

### siftDown(int index)

დანიშნულება: extractMax-ის შემდეგ root-ზე მოთავსებული ელემენტი ჩადის ქვემოთ სწორ ადგილამდე.

რას აკეთებს:

- იპოვის მარცხენა/მარჯვენა შვილებს.
- ირჩევს უფრო დიდი ხელფასის მქონე შვილს.
- თუ შვილზე პატარაა, ხდება swap და აგრძელებს ქვემოთ.

### გამოთვლითი სირთულე (Complexity)

Max-heap ოპერაციები მუშაობენ ლოგარითმულ დროში, რადგან heap-ის სიმაღლე არის  $O(\log n)$ .

- insert:  $O(\log n)$  — siftUp.
- peekMax:  $O(1)$  — Data[0].
- extractMax:  $O(\log n)$  — siftDown.
- print:  $O(n)$  — ყველა ელემენტის გავლა.

## თავი 7 — Driver.cpp

```
1  #include <iostream>
2  #include <string>
3  #include <random>
4  #include <ctime>
5
6  #include "PriorityQueue.h"
7  #include "WorkerClasses.h"
8
9  int main() {
10     PriorityQueue Q(70); // Priority queue that will store 70 employees (7 roles * 10 objects each)
11     int nextID = 1;      // Auto-incrementing employee ID generator
12
13     // Pool of random names used for generating employees
14     const std::string names[20] = {"Nika", "Giorgi", "Mariam", "Ana", "Luka", "Saba", "Nino", "Dato", "Salome", "Irakli",
15                                     "Tamar", "Elene", "Beka", "Keti", "Zurab", "Khatia", "Levan", "Tinatin", "Ilia", "Giga"};
16
17     // Random engine seeded for different results each run in [0;19] Range
18     std::default_random_engine dre((unsigned)time(NULL));
19     std::uniform_int_distribution<int> dis(0, 19);
20
21     // Create and insert 10 CIO employees
22     for (int i = 0; i < 10; ++i) {
23         Q.insert(new CIO(names[dis(dre)], nextID++, 12 + 3 * i));
24     }
25
26     // Create and insert 10 Project Managers
27     for (int i = 0; i < 10; ++i) {
28         Q.insert(new ProjectManager(names[dis(dre)], nextID++, 12 + 2 * i));
29     }
30
31     // Create and insert 10 Backend Developers
32     for (int i = 0; i < 10; ++i)
33     {
34         // Choose qualification level based on i
35         QualificationLevel lvl;
36         if (i % 3 == 0) lvl = QualificationLevel::Junior;
37         else if (i % 3 == 1) lvl = QualificationLevel::Middle;
38         else lvl = QualificationLevel::Senior;
39
40         // Choose backend tech based on i
41         DevTech tech;
42         if (i % 3 == 0) tech = DevTech::DotNet;
43         else if (i % 3 == 1) tech = DevTech::Spring;
44         else tech = DevTech::Django;
45
46         Q.insert(new BackendDeveloper(names[dis(dre)], nextID++, 6 + 2 * i, lvl, tech));
47     }
48
49     // Create and insert 10 Frontend Developers
50     for (int i = 0; i < 10; ++i)
51     {
52         QualificationLevel lvl;
```



```

49 // Create and insert 10 Frontend Developers
50 for (int i = 0; i < 10; ++i)
51 {
52     QualificationLevel lvl;
53     if (i % 3 == 0) lvl = QualificationLevel::Junior;
54     else if (i % 3 == 1) lvl = QualificationLevel::Middle;
55     else lvl = QualificationLevel::Senior;
56
57     // Choose frontend tech
58     DevTech tech;
59     if (i % 3 == 0) tech = DevTech::Angular;
60     else if (i % 3 == 1) tech = DevTech::React;
61     else tech = DevTech::Vue;
62
63     Q.insert(new FrontendDeveloper(names[dis(dre)], nextID++, 6 + 2 * i, lvl, tech));
64 }
65
66 // Create and insert 10 Database Engineers
67 for (int i = 0; i < 10; ++i)
68 {
69     QualificationLevel lvl;
70     if (i % 3 == 0) lvl = QualificationLevel::Junior;
71     else if (i % 3 == 1) lvl = QualificationLevel::Middle;
72     else lvl = QualificationLevel::Senior;
73
74     Q.insert(new DataBaseEngineer(names[dis(dre)], nextID++, 6 + 4 * i, lvl));
75 }
76
77 // Create and insert 10 DevOps Engineers
78 for (int i = 0; i < 10; ++i)
79 {
80     QualificationLevel lvl;
81     if (i % 3 == 0) lvl = QualificationLevel::Junior;
82     else if (i % 3 == 1) lvl = QualificationLevel::Middle;
83     else lvl = QualificationLevel::Senior;
84
85     Q.insert(new DevOpsEngineer(names[dis(dre)], nextID++, 6 + 4 * i, lvl));
86 }
87
88 // Create and insert 10 Testers
89 for (int i = 0; i < 10; ++i)
90 {
91     QualificationLevel lvl;
92     if (i % 3 == 0) lvl = QualificationLevel::Junior;
93     else if (i % 3 == 1) lvl = QualificationLevel::Middle;
94     else lvl = QualificationLevel::Senior;
95
96     Q.insert(new Tester(names[dis(dre)], nextID++, 6 + 3 * i, lvl));
97 }
98

```

```

99 // Print queue size and the heap content in heap order
100 std::cout << "\nTotal employees in priority queue: " << Q.getSize() << "\n\n";
101 Q.print(std::cout);
102
103 // Test peekMax()
104 if (!Q.isEmpty()) {
105     std::cout << "\n\nHighest paid employee (peekMax):\n";
106     Q.peekMax()->print(std::cout);
107 }
108
109 // Test extractMax(): remove top 5 highest paid employees
110 std::cout << "\n\nTop 5 employees by salary (extractMax):\n";
111 for (int i = 0; i < 5 && !Q.isEmpty(); ++i) {
112     Employee* E = Q.extractMax();
113     std::cout << i + 1 << " ";
114     E->print(std::cout);
115     std::cout << "\n";
116     delete E; // prevent memory leak for removed employees
117 }
118
119 // Print queue again after removals
120 std::cout << "\n\nQueue size after removing top 5: " << Q.getSize() << "\n\n";
121 Q.print(std::cout);
122 std::cout << "\n";
123
124 // Test isFull()
125 std::cout << "Is Queue Full? " << (Q.isFull() ? "YES" : "NO") << "\n";
126

```

## ზოგადი დანიშნულება

Driver.cpp არის ტესტირების პროგრამა: ქმნის მინიმუმ 10 ობიექტს თითოეული კონკრეტული კლასისთვის (ჯამში 70) და ეტაპობრივად ამატებს PriorityQueue-ში. შემდეგ ტესტავს ყველა მთავარ მეთოდს: insert, print, peekMax, extractMax, isEmpty, isFull, getSize.

## Q და nextID

PriorityQueue Q(70) — რიგის capacity არის 70, რადგან ზუსტად 7 პოზიციისთვის თითოეულზე ვქმნით 10 ობიექტს.

nextID — გენერატორია უნიკალური ID-ებისთვის. ყოველ ახალ თანამშრომელზე

## სახელების რანდომიზაცია

names[20] არის სახელების „pool“.

default\_random\_engine + uniform\_int\_distribution(0, 19) ირჩევს შემთხვევით ინდექსს და ასე გადანაწილდება სახელები თანამშრომლებზე.

## მეთოდების ტესტირება (რაც დავალებაში მოთხოვნილია)

getSize() — ბეჭდავს რამდენი თანამშრომელია რიგში.

print() — ბეჭდავს heap-ის შიდა მდგომარეობას.

peekMax() — აჩვენებს ყველაზე მაღალ ხელფასს (არ შლის).

extractMax() — იღებს ტოპ 5 თანამშრომელს ხელფასის მიხედვით და თითოეულზე აკეთებს delete-ს.

isEmpty() — გამოიყენება უსაფრთხო შემოწმებებისთვის, რომ nullptr არ მივიღოთ.

isFull() — ბოლოს ბეჭდავს რიგი სავსეა თუ არა.



# ტესტირების შედეგები

სიმარტივისთვის შევქმენი შვიდივე მიმართულების 2-2 ობიექტი და მათზე ჩავატარე ტესტი:

```
Total employees in priority queue: 14
```

```
[1] [CIO]
```

```
Name: Saba | ID: 2 | Class: CIO | Tenure(months): 15 | Base salary: 10000 | Total salary: 11500
```

```
[2] [CIO]
```

```
Name: Tamar | ID: 1 | Class: CIO | Tenure(months): 12 | Base salary: 10000 | Total salary: 11200
```

```
[3] [Project Manager]
```

```
Name: Elene | ID: 3 | Class: PM | Tenure(months): 12 | Base salary: 7000 | Total salary: 8080
```

```
[4] [Project Manager]
```

```
Name: Nino | ID: 4 | Class: PM | Tenure(months): 14 | Base salary: 7000 | Total salary: 8260
```

```
[5] [DB Engineer]
```

```
Name: Salome | ID: 10 | Class: DB | Tenure(months): 10 | Base salary: 6000 | Total salary: 6800 | Level: Middle
```

```
[6] [DevOps Engineer]
```

```
Name: Salome | ID: 12 | Class: DE | Tenure(months): 10 | Base salary: 6200 | Total salary: 7050 | Level: Middle
```

```
[7] [Tester]
```

```
Name: Ilia | ID: 14 | Class: TST | Tenure(months): 9 | Base salary: 4000 | Total salary: 4360 | Level: Middle
```

```
[8] [Frontend Dev]
```

```
Name: Salome | ID: 8 | Class: FD | Tenure(months): 8 | Base salary: 5450 | Total salary: 5970 | Level: Middle | Tech: React
```

```
[9] [DB Engineer]
```

```
Name: Saba | ID: 9 | Class: DB | Tenure(months): 6 | Base salary: 4500 | Total salary: 4980 | Level: Junior
```

```
[10] [Backend Dev]
```

```
Name: Mariam | ID: 5 | Class: BD | Tenure(months): 6 | Base salary: 4150 | Total salary: 4570 | Level: Junior | Tech: .Net
```

```
[11] [DevOps Engineer]
```

```
Name: Giga | ID: 11 | Class: DE | Tenure(months): 6 | Base salary: 4600 | Total salary: 5110 | Level: Junior
```

```
[12] [Backend Dev]
```

```
Name: Salome | ID: 6 | Class: BD | Tenure(months): 8 | Base salary: 5800 | Total salary: 6360 | Level: Middle | Tech: Spring
```

```
[13] [Tester]
```

```
Name: Ilia | ID: 13 | Class: TST | Tenure(months): 6 | Base salary: 3000 | Total salary: 3240 | Level: Junior
```

```
[14] [Frontend Dev]
```

```
Name: Salome | ID: 7 | Class: FD | Tenure(months): 6 | Base salary: 3950 | Total salary: 4340 | Level: Junior | Tech: Angular
```

```
Highest paid employee (peekMax):
```

```
[CIO]
```

```
Name: Saba | ID: 2 | Class: CIO | Tenure(months): 15 | Base salary: 10000 | Total salary: 11500
```

```
Top 5 employees by salary (extractMax):
```

```
1) [CIO]
```

```
Name: Saba | ID: 2 | Class: CIO | Tenure(months): 15 | Base salary: 10000 | Total salary: 11500
```

```
2) [CIO]
```

```
Name: Tamar | ID: 1 | Class: CIO | Tenure(months): 12 | Base salary: 10000 | Total salary: 11200
```

```
3) [Project Manager]
```

```
Name: Nino | ID: 4 | Class: PM | Tenure(months): 14 | Base salary: 7000 | Total salary: 8260
```

```
4) [Project Manager]
```

```
Name: Elene | ID: 3 | Class: PM | Tenure(months): 12 | Base salary: 7000 | Total salary: 8080
```

```
5) [DevOps Engineer]
```

```
Name: Salome | ID: 12 | Class: DE | Tenure(months): 10 | Base salary: 6200 | Total salary: 7050 | Level: Middle
```

Queue size after removing top 5: 9

[1] [DB Engineer]

Name: Salome | ID: 10 | Class: DB | Tenure(months): 10 | Base salary: 6000 | Total salary: 6800 | Level: Middle

[2] [Frontend Dev]

Name: Salome | ID: 8 | Class: FD | Tenure(months): 8 | Base salary: 5450 | Total salary: 5970 | Level: Middle | Tech: React

[3] [Backend Dev]

Name: Salome | ID: 6 | Class: BD | Tenure(months): 8 | Base salary: 5800 | Total salary: 6360 | Level: Middle | Tech: Spring

[4] [DB Engineer]

Name: Saba | ID: 9 | Class: DB | Tenure(months): 6 | Base salary: 4500 | Total salary: 4980 | Level: Junior

[5] [DevOps Engineer]

Name: Giga | ID: 11 | Class: DE | Tenure(months): 6 | Base salary: 4600 | Total salary: 5110 | Level: Junior

[6] [Tester]

Name: Ilia | ID: 13 | Class: TST | Tenure(months): 6 | Base salary: 3000 | Total salary: 3240 | Level: Junior

[7] [Tester]

Name: Ilia | ID: 14 | Class: TST | Tenure(months): 9 | Base salary: 4000 | Total salary: 4360 | Level: Middle

[8] [Frontend Dev]

Name: Salome | ID: 7 | Class: FD | Tenure(months): 6 | Base salary: 3950 | Total salary: 4340 | Level: Junior | Tech: Angular

[9] [Backend Dev]

Name: Mariam | ID: 5 | Class: BD | Tenure(months): 6 | Base salary: 4150 | Total salary: 4570 | Level: Junior | Tech: .Net

Is Queue Full? NO

C:\Users\iakob\source\repos\Employee\x64\Debug\Employee.exe (process 53404) exited with code 0 (0x0).

Press any key to close this window . . .